

Déploiement production d'ETIGE Manager

Ce document décrit le déploiement d'ETIGE Manager pour une entreprise d'environ 10 utilisateurs, avec accès depuis le bureau et le domicile.

Architecture recommandée :

```
Nom de domaine (.com ou .ci)
|
HTTPS avec Let's Encrypt
|
Nginx -> Gunicorn -> Django
|
PostgreSQL + fichiers media/ (photos et documents)
|
Sauvegardes externalisées
```

1. Budget estimatif en FCFA

Les prix changent selon le fournisseur, les promotions, la TVA et le taux de change. Ce sont des ordres de grandeur à vérifier au moment de l'achat.

Option recommandée pour 10 utilisateurs

Élément	Coût estimatif
VPS Linux 2 vCPU, 4 Go RAM, 80 Go SSD	35 000 à 70 000 FCFA/an
Nom de domaine .com	8 000 à 15 000 FCFA/an
Nom de domaine .ci	25 000 à 60 000 FCFA/an
Certificat HTTPS Let's Encrypt	0 FCFA
Stockage sauvegardes 100 à 250 Go	15 000 à 50 000 FCFA/an
Emails professionnels, optionnels	25 000 à 100 000 FCFA/an
Mise en place technique, si sous-traitée	150 000 à 500 000 FCFA une fois

Totaux approximatifs

- Avec un domaine .com : 58 000 à 135 000 FCFA/an, hors emails et prestation.
- Avec un domaine .ci : 75 000 à 180 000 FCFA/an, hors emails et prestation.
- Avec .com et .ci : acheter les deux n'est pas obligatoire. Un seul domaine suffit pour commencer.

Le domaine .ci est intéressant pour affirmer l'identité ivoirienne. Le .com est souvent plus simple et moins cher. Une configuration possible est :

`https://manager.entreprise.ci`

et réserver aussi :

`https://entreprise.com`

Le domaine non principal peut rediriger vers le domaine principal.

2. Ce qu'il faut acheter

Créer les éléments suivants :

Un VPS Ubuntu 24.04 LTS chez un fournisseur cloud.

Un nom de domaine .com ou .ci.

Un espace de sauvegarde externe compatible S3, Backblaze, Wasabi ou équivalent.

Éventuellement des adresses email professionnelles.

Caractéristiques minimales du VPS

Pour 10 utilisateurs et l'application actuelle :

- 2 vCPU ;
- 4 Go de RAM ;
- 80 Go SSD minimum ;
- Ubuntu 24.04 LTS ;

- adresse IPv4 publique ;
- sauvegardes ou snapshots proposés par le fournisseur.

Ne pas utiliser un hébergement mutualisé classique : Django, PostgreSQL, Gunicorn et les fichiers médias nécessitent un serveur contrôlable.

3. Préparer le serveur

Se connecter au serveur avec le compte fourni par l'hébergeur :

```
ssh root@IP_DU_SERVEUR
```

Mettre le système à jour :

```
apt update && apt upgrade -y
apt install -y nginx postgresql postgresql-contrib python3 python3-venv python3-pip python3-dev build-essential git curl unzip
```

Créer un utilisateur système dédié :

```
adduser --disabled-password --gecos "" etige
usermod -aG sudo etige
mkdir -p /srv/etige-manager
chown -R etige:etige /srv/etige-manager
```

Se connecter avec cet utilisateur :

```
su - etige
cd /srv/etige-manager
```

4. Installer le projet

Transférer le projet depuis le poste de développement, ou le récupérer depuis Git :

```
git clone URL_DU_DEPOT /srv/etige-manager
cd /srv/etige-manager
```

Ne jamais transférer .env, les mots de passe ou les clés secrètes dans un dépôt public.

Créer l'environnement Python :

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Installer Node.js 20 ou plus récent, puis construire le frontend :

```
npm install
npm run build
```

5. Configurer PostgreSQL

Ouvrir PostgreSQL :

```
sudo -u postgres psql
```

Créer un utilisateur et une base avec un mot de passe long et unique :

```
CREATE USER etige_db_user WITH PASSWORD 'REPLACER_PAR_UN_MOT_DE_PASSE_FORT';
CREATE DATABASE etige_manager OWNER etige_db_user;
\q
```

Le mot de passe ne doit pas être écrit dans ce document ni envoyé dans un chat.

6. Configurer les variables d'environnement

Créer le fichier de production :

```
cd /srv/etige-manager
nano .env
```

Exemple à adapter :

```
DJANGO_SECRET_KEY=GENERER_UNE_CLE_LONGUE_ET_ALEATOIRE
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=manager.entreprise.ci

POSTGRES_DB=etige_manager
POSTGRES_USER=etige_db_user
POSTGRES_PASSWORD=MOT_DE_PASSE_POSTGRESQL
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
```

Protéger le fichier :

```
chmod 600 .env
chown etige:etige .env
```

Générer une clé Django avec Python :

```
.venv/bin/python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

7. Appliquer les migrations et préparer les fichiers

```
source /srv/etige-manager/.venv/bin/activate
cd /srv/etige-manager
python manage.py migrate
python manage.py setup_roles
python manage.py collectstatic --noinput
python manage.py check --deploy
```

Le projet utilise :

- staticfiles/ pour les fichiers compilés et statiques ;
- media/ pour les photos importées et autres fichiers utilisateurs.

Le dossier media/ doit être sauvegardé. Il ne faut jamais le supprimer pendant une mise à jour.

8. Créer ou conserver le compte administrateur

Si le compte superutilisateur existe déjà dans la base transférée, ne pas en créer un nouveau.

Sinon :

```
python manage.py createsuperuser
```

Il doit y avoir un seul compte superutilisateur pour l'administrateur technique. Les autres comptes sont des utilisateurs normaux :

- DG ;
- Directeur technique ;
- Manager.

Ils ne doivent pas avoir is_staff=True ni is_superuser=True, sauf décision explicite de l'entreprise.

Les groupes sont créés avec :

```
python manage.py setup_roles
```

Les groupes DG, Directeur technique et Manager servent à identifier les rôles. Les permissions fines doivent encore être renforcées avant de considérer le système comme complètement cloisonné.

9. Installer Gunicorn

Le fichier requirements.txt actuel ne contient pas encore Gunicorn. L'installer sur le serveur :

```
source /srv/etige-manager/.venv/bin/activate
pip install gunicorn
```

Pour rendre l'installation reproductible, ajouter ensuite la dépendance Gunicorn dans requirements.txt avant le prochain déploiement :

```
gunicorn>=23.0
```

Tester Gunicorn :

```
cd /srv/etige-manager
.venv/bin/gunicorn --bind 127.0.0.1:8000 config.wsgi:application
```

Laisser ce test actif dans un terminal séparé, puis vérifier localement :

```
curl http://127.0.0.1:8000/
```

Arrêter avec Ctrl+C.

10. Créer le service systemd

Créer le fichier :

```
sudo nano /etc/systemd/system/etige-manager.service
```

Contenu :

```
[Unit]
Description=ETIGE Manager Django
After=network.target postgresql.service

[Service]
User=etige
Group=etige
WorkingDirectory=/srv/etige-manager
EnvironmentFile=/srv/etige-manager/.env
ExecStart=/srv/etige-manager/.venv/bin/gunicorn --workers 3 --bind 127.0.0.1:8000 config.wsgi:application
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activer et démarrer :

```
sudo systemctl daemon-reload
sudo systemctl enable etige-manager
sudo systemctl start etige-manager
sudo systemctl status etige-manager
```

Voir les erreurs :

```
sudo journalctl -u etige-manager -f
```

11. Configurer Nginx

Remplacer manager.entreprise.ci par le domaine choisi :

```
sudo nano /etc/nginx/sites-available/etige-manager
```

Configuration :

```
server {
    listen 80;
    server_name manager.entreprise.ci;

    client_max_body_size 25M;

    location /static/ {
        alias /srv/etige-manager/staticfiles/;
    }

    location /media/ {
        alias /srv/etige-manager/media/;
    }

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Activer la configuration :

```
sudo ln -s /etc/nginx/sites-available/etige-manager /etc/nginx/sites-enabled/etige-manager
sudo nginx -t
sudo systemctl reload nginx
```

12. Configurer le domaine .com ou .ci

Dans le panneau du registrar, créer un enregistrement DNS :

```
Type : A
Nom : manager
Valeur : IP_DU_SERVEUR
TTL : 300 ou automatique
```

Pour un domaine .com, l'adresse sera par exemple :

```
manager.entreprise.com
```

Pour un domaine .ci :

```
manager.entreprise.ci
```

Attendre la propagation DNS, généralement quelques minutes à 24 heures. Vérifier depuis le serveur :

```
dig +short manager.entreprise.ci
```

Le résultat doit afficher l'adresse IP du VPS.

13. Activer HTTPS gratuitement

Installer Certbot :

```
sudo apt install -y certbot python3-certbot-nginx
```

Obtenir le certificat :

```
sudo certbot --nginx -d manager.entreprise.ci
```

Choisir la redirection automatique HTTP vers HTTPS lorsque Certbot le propose.

Tester le renouvellement :

```
sudo certbot renew --dry-run
```

Après activation, l'application sera accessible avec :

```
https://manager.entreprise.ci
```

Le certificat Let's Encrypt coûte 0 FCFA. Le domaine et le serveur restent payants.

14. Pare-feu du serveur

Autoriser uniquement SSH, HTTP et HTTPS :

```
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

Ne jamais exposer PostgreSQL sur Internet. Le port 5432 doit rester fermé publiquement.

15. Sauvegardes PostgreSQL

Créer un dossier de sauvegarde temporaire :

```
sudo mkdir -p /var/backups/etige-manager
sudo chown etige:etige /var/backups/etige-manager
```

Créer un script :

```
nano /srv/etige-manager/backup.sh
chmod 700 /srv/etige-manager/backup.sh
```

Contenu :

```
#!/usr/bin/env bash
set -euo pipefail

DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_DIR=/var/backups/etige-manager
DB_FILE="$BACKUP_DIR/database_${DATE}.sql.gz"
MEDIA_FILE="$BACKUP_DIR/media_${DATE}.tar.gz"

PGPASSWORD='MOT_DE_PASSE_POSTGRESQL' pg_dump -h 127.0.0.1 -U etige_db_user etige_manager | gzip > "$DB_FILE"
tar -czf "$MEDIA_FILE" -C /srv/etige-manager media

find "$BACKUP_DIR" -type f -mtime +14 -delete
```

Pour une vraie sécurité, ne pas laisser les seules sauvegardes sur le VPS. Copier automatiquement les fichiers vers un stockage externe chiffré.

Ajouter une tâche quotidienne :

```
crontab -e

15 2 * * * /srv/etige-manager/backup.sh >> /var/log/etige-manager-backup.log 2>&1
```

Politique recommandée :

- sauvegarde PostgreSQL quotidienne ;
- sauvegarde du dossier media/ quotidienne ;
- conservation locale de 14 jours ;
- copie externe de 30 à 90 jours ;
- test de restauration au moins une fois par mois.

16. Déploiement des mises à jour

Depuis le serveur :

```
cd /srv/etige-manager
sudo systemctl stop etige-manager
cp -a media /tmp/etige-media-backup
source .venv/bin/activate
git pull
pip install -r requirements.txt
npm install
npm run build
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py check --deploy
sudo systemctl start etige-manager
sudo systemctl reload nginx
```

Vérifier le service :

```
sudo systemctl status etige-manager
sudo journalctl -u etige-manager -n 100 --no-pager
```

Ne jamais lancer makemigrations en production pour créer une migration improvisée. Les migrations doivent être créées et testées dans le dépôt avant d'être appliquées avec migrate.

17. Vérification finale avant livraison du lien

Tester les points suivants avec le domaine HTTPS :

- connexion administrateur ;
- connexion DG ;
- connexion Directeur technique ;
- connexion Manager ;
- création d'un projet ;
- modification d'un projet ;
- ajout de photos Survey ;
- affichage des photos dans la fiche projet ;
- ajout de photos de clôture ;
- création d'un devis avec plusieurs lignes ;
- téléchargement du PDF ;
- suppression contrôlée d'un projet ;
- accès à /admin/ uniquement pour le superutilisateur ;
- renouvellement HTTPS ;
- restauration d'une sauvegarde de test.

Lien à transmettre aux utilisateurs :

`https://manager.entreprise.ci`

Ne jamais transmettre /admin/ aux utilisateurs normaux.

18. Points de sécurité avant ouverture publique

Avant la mise en ligne réelle :

- vérifier que DJANGO_DEBUG=False ;
- vérifier que DJANGO_SECRET_KEY n'est pas la valeur d'exemple ;
- ne jamais commiter .env ;
- ne jamais afficher de mot de passe dans les logs ;
- utiliser des mots de passe uniques ;
- activer HTTPS et la redirection HTTP vers HTTPS ;
- garder PostgreSQL inaccessible depuis Internet ;
- limiter les permissions par rôle ;
- protéger les sauvegardes ;
- mettre à jour Ubuntu, Python, Django et les dépendances ;
- surveiller les logs sans y mettre de données sensibles ;
- prévoir une procédure de récupération du compte administrateur.

La version actuelle possède les groupes DG, Directeur technique et Manager, mais les permissions métier fines doivent encore être définies et appliquées si chaque rôle doit avoir des droits différents.

19. Résultat attendu

À la fin du déploiement :

- l'application tourne sur le VPS ;
- PostgreSQL stocke les données ;
- Nginx sert les fichiers statiques et les photos ;
- Gunicorn lance Django ;
- HTTPS protège les connexions ;
- le domaine .com ou .ci permet l'accès depuis le bureau et la maison ;
- les utilisateurs normaux travaillent sans droits d'administration ;

- les sauvegardes protègent la base et les photos.